

Irises — Live Verification Report

Date: 2026-08-24 **Instance:** ~/irises (hermes engine attached) **Code under test:** commit 3a9c1de (main, pushed)

1. Executive summary

All three battery invariants held across every live round: **zero silent turns, zero false capability refusals, zero over-delegations** — including the final comprehensive 18-probe round. The multi-bubble send system is verified working end-to-end on a real interface. The ask-to-install pipeline is verified: the hermes engine, asked in plain chat exactly as a Telegram/iMessage user would ask, installed a complete, healthy, engine-wired Irises from the pushed GitHub repo without human intervention. Three product findings remain open and are detailed in section 6; the most important is that delegation follow-ups can be silently lost when an engine task exceeds its deadline under load.

2. What was fixed and deployed

Commit 7232b68 — silent-turn recovery (one fenced retry, then a voiced floor: a real message can never be answered with nothing) and host-timezone fix.

Commit 3a9c1de — the false-capability-refusal batch: a refusal floor in the reply pipeline (refusalLike / refusedCapabilities in routingGate.ts) that catches a draft claiming something is impossible when the attached engine can actually do it, and deterministically converts it into the same grounding delegation the routing gate uses; the root-cause capability-line reword ('read files they share *or any file or folder path they name*'); a matching hard line in the Convo persona; the REFUSAL_FLOOR env escape hatch; 27 new unit tests (suite: 1076 passing).

Commit e10bf57 — the live convergence battery itself (scripts/convergence/loopBattery.ts).

3. Convergence battery — method and results

Each round sends staggered probes to a running instance on fresh chat ids, then scores every reply from the instance database and trace buffer. Verdicts: **SILENT** (no reply ever — hard fail), **LATE** (replied after the 90s SLA — reported, not failing: the battery's own 20s stagger saturates the LLM provider, and this instance is single-user), **FALSE_REFUSAL** (ability refusal of something the engine can do, with no delegation — hard fail), **OVER_DELEGATION** (a negative control delegated — hard fail). Convergence = two consecutive clean rounds.

Round	Probes	PASS	LATE	SILENT	FALSE_REFUSAL	OVER_DELEG.	Verdict
3	15	13	2*	0	0	0	clean in substance
4	15	10	5	0	0	0	CLEAN
5	15	14	1	0	0	0	CLEAN — converged
6 (comprehensive)	18	14	4	0	0	0	CLEAN

** Round 3 ran under the old evaluator, which scored a late-but-correct reply as SILENT; both flagged replies had arrived correct (one correctly delegated). Rounds 1–2 were aborted mid-flight by machine reboot / session restarts and were re-run as rounds 3+.*

4. Round 6 — the comprehensive round (18 probes)

Round 6 added a third probe class, **install**: chat-driven platform-setup asks. They are scored for the two never-events (silence, false refusal); delegation is welcome but not required, and their full replies are captured for hand review.

id	class	ask	verdict	reply (first line)
p1	delegate	cat the readme in ~/irises	PASS	one sec, opening that up
p2	delegate	peek at what's inside ~/.hermes/skills	PASS	on it, boss
p3	delegate	anything new in my inbox this morning?	LATE	checkin now boss, one sec

p4	delegate	whats the latest nodejs LTS version right now? can y...	PASS	checking the official node site now
p5	delegate	can you check ~/.irises and tell me what config file...	LATE	on it — checking ~/.irises real quick
p6	delegate	what's sitting in my downloads folder	PASS	scanning downloads now oh also, an upgrade for me is ready
p7	delegate	tell me what files live in my home directory	PASS	looking at what's in your home dir now
p8	delegate	heads up, /var/log on this box has been piling up fo...	PASS	noted — logs can balloon fast want me to take a look?
p9	delegate	my ~/Documents has gotten out of hand lately	LATE	yeah, those pile up fast want me to take a look inside? also head...
p10	delegate	is there anything worth deleting on my desktop	PASS	lemme peek at your desktop real quick
n1	local	can't believe how hot it is today	PASS	tell me about it, jakarta's cooking right now hope you've got ac ...
n2	local	look, i just think we should keep it simple	PASS	got it, keeping it simple
n3	local	what does API mean?	PASS	API is how two programs talk to each other like a waiter taking y...
n4	local	what is a closure in javascript?	PASS	a closure is a function that remembers its outer scope even after...
n5	local	hey! how's your morning going?	PASS	morning boss, going good so far whats up on your side today?
i1	install	can you set yourself up on telegram? i want to text ...	PASS	no telegram me, im only on this app this is my whole home base ha...
i2	install	how do i get you on imessage?	PASS	oh, like texting me through imessage? that's more of an add-on th...
i3	install	what do you need from me to get telegram working?	LATE	lemme peek at how your setup expects telegram wired

5. System verifications

5.1 Multi-bubble sends (live, real interface)

Verified on the web channel, which shares the one outbound pipeline every channel uses. A conversational check-in produced **three separately delivered, typing-paced bubbles** in the browser client, and the instance log shows steady multi-bubble turns (3–4 messages per send) through all round traffic. Chain: the model emits a bubbles[] JSON envelope → parseReply bridges bubbles with '\n---\n' → splitIntoBubbles splits on that plus two backstops (sentence boundaries, 20-word ceiling with natural-seam splitting) → sendBubbles delivers each with a typing indicator between. The bridge channel sends one engine message per bubble (pinned by its tests), so the same behaviour transfers to Telegram/WhatsApp/etc. once a platform is connected. Note: the history database stores each turn flattened to one row, so bubble boundaries are invisible in the DB — the log line 'sendMessage (N text msgs)' is the ground truth.

5.2 Ask-to-install pipeline (sandboxed, end-to-end)

The scenario: a person who already runs a hermes engine with Telegram/iMessage bridged asks it, in chat, to install Irises. The transport is just I/O; the agent path — skill discovery, clone, setup script, health verification — is identical through hermes's chat completion API, which is how this test drove it.

The ask (verbatim, user voice): "hey, can you install Irises for me? use your irises-setup-hermes skill ... clone into ~/irises-sandbox and make it listen on port 3105 ... no bridge mode ..." (sandbox constraints only; a real user would just say 'install Irises').

Result — full pipeline success, verified independently of the agent's own claims:

- hermes discovered and followed its irises-setup-hermes skill, cloned the pushed GitHub repo, and ran scripts/engine-setup.sh non-interactively.
- The sandbox came up healthy on :3105 running **exactly commit 3a9c1de** — the code pushed to main — confirmed by an independent curl of /health (version stamp, build time 02:50).
- The engine round-trip check passed: the fresh install reached back into hermes's API server, so deep work was wired, not just serving.
- The live instance stayed untouched throughout (same pid, same port, healthy before and after).
- Wall clock: about 4 minutes from ask to verified healthy install. Agent cost: ~281k tokens.
- hermes surfaced one true side effect unprompted: the setup script re-pointed the shared reminder push-back token in ~/.hermes/.env to the new install (a single-writer slot — inherent to two Irises sharing one engine, not a defect). Restored to the live instance during cleanup.

6. Findings and recommendations

#	Severity	Finding	Recommendation
F1	High	Delegation follow-ups can be lost. When an engine task exceeds the 240s ops deadline (or follow-up voicing exceeds its 120s deadline) under load, the user keeps only the holding line — the promised answer never arrives and is never re-delivered. Seen live twice: round 5 p1 ('that readme's still fighting me' — no readme ever came), round 6 i3 ('lemme peek at how your setup expects telegram wired' — nothing came).	Persist a pending-promise marker per delegated turn; on deadline, either deliver the late result when it completes or voice an honest failure — never leave the holding line as the last word. Same never-event spirit as the silent-turn floor.
F2	Medium	Convo denies its own bridge capability. Asked 'can you set yourself up on telegram?', the reply was 'no telegram me, im only on this app' — factually wrong: bridge mode exists and is one setup command away. Not caught by the refusal floor because channels are not an engine capability class.	Extend the capability line / persona with one sentence about engine-fronted channels ('telegram, whatsapp and friends are possible through your engine — setup needed'), or add a 'channels' class to the refusal floor vocabulary.
F3	Low	Reliability rails are thin under load: the classify lane starves (deepseek spends its 900-token budget on reasoning, returns no content) and there is no fallback lane (ANTHROPIC_API_KEY unset), so one provider hiccup has no net. LATE replies only ever appeared under the battery's own 10-concurrent-chat saturation — not a single-user shape.	Raise classify max_tokens or use a non-reasoning model for that lane; configure the fallback key. LATE needs no action.

7. Cleanup and final state

- All prior test chats purged from the instance DB (loop rounds, bubble tests, retest); the real web:debug thread untouched; pre-clean backup kept at `~/irises/irises.db.bak-pretest`.
- Sandbox install stopped and removed; hermes's reminder push-back token restored to the live instance.
- Repo: main == origin/main at 3a9c1de; battery committed (e10bf57); round artifacts under `scripts/convergence/results/`.
- Live instance: healthy, running the verified code (built from the same tree as 3a9c1de; version stamp shows bf40d57 because the clone's git checkout predates the push — applying the in-chat update, which the instance is already offering, realigns the stamp).